

# Foundations of Quantum Computing

## Week 1: Boolean logic, reversible computation, complexity and randomised computation

### Explore – Study

This week you will be introduced to some of the foundational methods of quantum computation. In this week’s Study section you will cover the following topics:

1.1 Boolean logic

1.2 Reversible computation

1.3 Randomised computation

### 1.1 Boolean logic

‘Classical’ computing (the kind done by smartphones and supercomputing clusters alike) involves computations using 0s and 1s. We usually care about things other than 0s and 1s in a computation, such as numbers (integers or fractions), written characters, colours and so forth. But in each case, we can only work in practice with some **finite** number  $N$  of possible values for these quantities – a range of possibilities which may be large, but which could in principle be counted. Computers usually represent such finite ranges  $A = \{a_0, a_1, \dots, a_{N-1}\}$  of possibilities using integers (using ‘0’ to represent  $a_0$ , ‘1’ to represent  $a_1$ , and so forth). These integers are then represented through ‘binary encoding’, using 0s and 1s – and all computations involving those integers are done using what is known as **boolean logic**.

We will start by considering how data is represented in binary, and how computation is done on individual bits, to help us understand computation at a low level.

#### 1.1.1 Binary encoding and bitstrings

A single bit is a value  $x \in \{0, 1\}$ . We may occasionally write a sequence of  $n$  bits as a list / array / vector of bits  $x = (x_1, x_2, \dots, x_n)$ , but most often we will write it without any delimiters as  $x_1 x_2 \dots x_n$  (for instance, 001 or 10110). We refer to this as a ‘bitstring’, or an ‘ $n$ -bit string’ in the case that we want to make clear the length  $n$  of the sequence. We write the set of  $n$ -bit strings, as

$$\{0, 1\}^n = \{x_1 x_2 \dots x_n \mid x_1, x_2, \dots, x_n \in \{0, 1\}\} \quad (1.1)$$

using an exponential-like notation. (This notation is similar to the notation  $\mathbb{R}^n$ , for a vector given by  $n$  real coefficients.)

The number of possible  $n$ -bit strings – the number of elements in the set  $\{0, 1\}^n$  – grows exponentially: it's  $2^n$ , which is the result of multiplying by 2 a total of  $n$  times. This is a fast-growing quantity: we already have  $2^n > 50$  for  $n = 6$ , and  $2^n > 1000$  for  $n = 10$ , and  $2^n > 1\,000\,000$  for  $n = 20$ . We can then represent **large** finite sets, including large ranges of numbers, by ‘encoding’ them as  $n$ -bit strings for a large enough  $n$ .

To represent non-negative integers using bitstrings, we use **binary** (or ‘base-2’) representation. This is very much like the usual decimal notation for numbers, but instead of a ones ( $1 = 10^0$ ) place, a tens ( $10 = 10^1$ ) place, a hundreds ( $100 = 10^2$ ) place, a thousands ( $1000 = 10^3$ ) place and so forth, we represent numbers with a one’s ( $1 = 2^0$ ) place, a two’s ( $2 = 2^1$ ) place, a four’s ( $4 = 2^2$ ) place, an eight’s ( $8 = 2^3$ ) place, and so forth. For instance, just as we can represent one thousand and fifty nine in decimal by

$$1059 = (1 \times 10^3) + (0 \times 10^2) + (5 \times 10^1) + (9 \times 10^0) = 1000 + 50 + 9, \quad (1.2)$$

we can represent eleven in binary by

$$1011 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 8 + 2 + 1. \quad (1.3)$$

When we use a bitstring  $x \in \{0, 1\}^n$  to represent an integer in binary notation, we use a different convention for indexing the individual bits. Instead of writing  $x = x_1 x_2 \cdots x_n$ , we write  $x = x_{n-1} x_{n-2} \cdots x_2 x_1 x_0$ . The reason for this, is so that the bit  $x_k$  represents the  $2^k$ -place. So, for 1011 above, we would write  $x_3 = 1$ ,  $x_2 = 0$ ,  $x_1 = 1$  and  $x_0 = 1$ .

The smallest number that we can represent using  $n$  bits in this way is always zero, represented by  $00 \cdots 00 = (0 \times 2^{n-1}) + (0 \times 2^{n-2}) + \cdots + (0 \times 2^1) + (0 \times 2^0) = 0$ . The largest number we can represent with  $n$  bits is  $2^n - 1$ , represented by  $11 \cdots 11$ . So, an  $n$ -bit string  $x_{n-1} x_{n-2} \cdots x_2 x_1 x_0$  can represent any integer  $0 \leq n \leq 2^n - 1$ , using binary notation. (For this reason, among others, computer scientists often index their sequences, arrays and other lists starting from 0.)

In this way, we represent integers by bitstrings. For a finite set  $A = \{a_0, a_1, \dots, a_{M-1}\}$ , we may then represent the elements of  $A$  by bitstrings, by considering an integer  $m > 0$  such that  $M \leq 2^m$ , and using the  $m$ -bit strings representing 0, 1, and so forth up to  $M - 1$  to represent the elements of  $A$ . (There may be additional strings between  $M - 1$  and  $2^m - 1$  which do not represent any element of  $A$ : we don’t mind that.)

### 1.1.2 Functions on binary encodings

When we want to compute a function  $f : A \rightarrow B$  between two sets  $A = \{a_0, a_1, \dots, a_{M-1}\}$  and  $B = \{b_0, b_1, \dots, b_{N-1}\}$ , we would represent the possible inputs  $a \in A$  by bitstrings  $x = x_1 x_2 \cdots x_{m-1} \in \{0, 1\}^m$ , and possible outputs  $b \in B$  by bitstrings  $y = y_1 y_2 \cdots y_{n-1} \in \{0, 1\}^n$ . The function  $f$  can then be realised in one or more ways as a function

$$F(x_1 x_2 \cdots x_m) = y_1 y_2 \cdots y_n. \quad (1.4)$$

One way in which we can do this – which won’t always be the most efficient, but which is always possible – is by considering how each bit  $y_1$ ,  $y_2$ , and so forth can **independently**

be computed from the bitstring  $x \in \{0, 1\}^m$ . We can consider  $n$  functions

$$F_1(x_1x_2\cdots x_m) = y_1, \quad F_2(x_1x_2\cdots x_m) = y_2, \quad \dots, \quad F_n(x_1x_2\cdots x_m) = y_n. \quad (1.5)$$

Then, any function  $f : A \rightarrow B$  that you might want to compute on finite sets, can be expressed in terms of (one or more) functions  $f : \{0, 1\}^m \rightarrow \{0, 1\}$ , from a bitstring of some length to a single bit of output.

In this way, all practical computations can be described using computations taking  $m$  bit-values  $x_1, x_2, \dots, x_m \in \{0, 1\}$  as input, and producing a single-bit outcome  $y \in \{0, 1\}$  as output. We will now consider how such computations are done.

### 1.1.3 Logical operators

Using binary encodings as described above, computation ultimately boils down to boolean logic: definitions of functions on bitstrings, using operators  $\neg x$ ,  $x \& y$ , and  $x \vee y$  on values  $x, y, z, \dots \in \{0, 1\}$ . The effect of these operations are described in Table 1.1. We call ‘ $\neg$ ’ the ‘NOT’ operator, ‘ $\&$ ’ the ‘AND’ operator, and ‘ $\vee$ ’ the ‘OR’ operator. For us, it will very useful to consider a fourth operation,  $A \oplus B$ , where ‘ $\oplus$ ’ is the ‘exclusive-or’, ‘XOR’, or ‘addition mod 2’ operator.

Table 1.1: ‘Truth-tables’ for four boolean logical operators.

| $x, y$       |   |   | 0, 0 | 0, 1 | 1, 0 | 1, 1 |
|--------------|---|---|------|------|------|------|
| $x$          | 0 | 1 |      |      |      |      |
| $\neg x$     | 1 | 0 |      |      |      |      |
| $x \& y$     | 0 | 0 | 0    | 0    | 0    | 1    |
| $x \oplus y$ | 0 | 1 | 1    | 1    | 0    | 0    |
| $x \vee y$   | 0 | 1 | 1    | 1    | 1    | 1    |

The tables in Table 1.1 can be used as lookup-tables, with the input values of the operands running along the top, and the operation to be performed running along the column. The table on the left describes how the negation operation ‘ $\neg$ ’ acts on the two possible inputs. The other table describe how the boolean operators ‘ $\&$ ’, ‘ $\vee$ ’ and ‘ $\oplus$ ’ act on pairs of inputs.

We may define these by considering their values for all possible combinations of values of  $A$  and  $B$  as shown in Table 1.1. The values of the formulas can also be described using algebraic formulae:

$$\begin{aligned} \neg A &:= 1 - A, & A \vee B &:= A + B - AB \\ A \& B &:= AB & A \oplus B &:= A + B - 2AB. \end{aligned} \quad (1.6)$$

Here,  $AB$  just means the result of multiplying  $A$  and  $B$  as numbers. In this way, we can describe logical operations on truth-claims using algebra.

Figure 1.1: A sketch of a function on boolean strings, which evaluates to 1 for exactly one input. One way in which a boolean function  $f$  on  $n$  bits could be represented is as a bar chart, listing all possible inputs from left to right in order of the integers from 0 to  $2^n - 1$ , with a bar of height 1 for inputs  $x$  such that  $f(x) = 1$ , and a bar of height 0 for all other inputs. In this figure, we illustrate a hypothetical function for which  $f(x) = 1$  only for some specific value  $x = a_j$ .

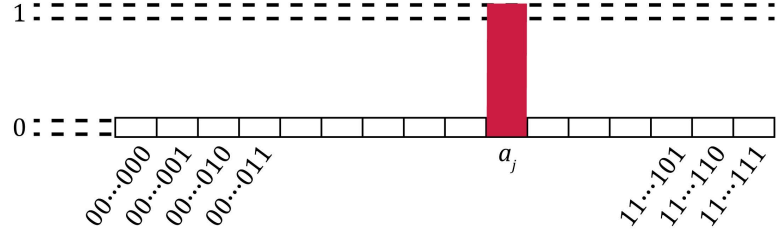


Figure 1.1. Long description: A bar chart. The horizontal axis is labeled by binary strings of unspecified length, starting with  $00 \dots 000$ ,  $00 \dots 001$ ,  $00 \dots 010$ ,  $00 \dots 011$ , in order of the integers 0, 1, 2, 3 that these would represent. The labeling of the  $x$  axis pauses for several spaces along the axis, and begins again on the right hand side with  $11 \dots 101$ ,  $11 \dots 110$ ,  $11 \dots 111$ , representing the final few  $n$ -bit strings in this ordering. The vertical axis simply has the two boolean values of 0 and 1. In the otherwise unlabelled space on the horizontal axis, there is a single position labeled by ' $a_j$ ', meant to represent some generic  $n$ -bit string. The bar at this location has height 1. The entire rest of the chart has bars of height 0.

#### 1.1.4 Universality of boolean logic

One of the most important results about computation and classical logic, is that **every** function  $f : \{0, 1\}^m \rightarrow \{0, 1\}$  may be broken down into simpler computations, consisting of exactly these operations. This result is often described as '**the universality of boolean logic**'. Together with the observation that we made above about binary encoding, this means that we can reduce any computation on finite sets, into extremely simple computations which we can realise using simple reproducible parts.

To demonstrate this for a function  $f : \{0, 1\}^m \rightarrow \{0, 1\}$ , consider the different inputs  $\{a_1, a_2, \dots, a_r\} \subseteq \{0, 1\}^m$  for which  $f(a_j) = 1$ . There may be only a few such inputs, or there may be many. We will start, for any given one of these inputs  $a_j$ , by considering a boolean function  $\delta_{a_j} : \{0, 1\}^m \rightarrow \{0, 1\}$  such that  $\delta_{a_j}(x) = 1$  if and only if  $x = a_j$ . (See Figure 1.1 for an illustration.)

To describe a function  $\delta_a(x)$  which evaluates to 1 precisely for some value  $x = a \in \{0, 1\}^m$ , we can consider the special case of the formula  $x_1 \& x_2 \& \dots \& x_m$ . (Because  $(x_1 \& x_2) \& x_3$  is always equal to  $x_1 \& (x_2 \& x_3)$ , we may drop the parentheses and write them both as  $x_1 \& x_2 \& x_3$ . We may do the same for taking the 'AND' of more than three bits; similar remarks also apply to the 'OR' and the 'XOR' of three or more bits together.) For instance, the formula  $x_1 \& x_2 \& x_3$  represents an expression which is the same as

$\delta_{111}(x)$ , acting on three-bit strings  $x \in \{0,1\}^3$  as seen in Table 1.2.

Table 1.2: A truth-table for a boolean formula realising the function  $\delta_{111}$ .

| $x_1, x_2, x_3$     | 0,0,0 | 0,0,1 | 0,1,0 | 0,1,1 | 1,0,0 | 1,0,1 | 1,1,0 | 1,1,1 |
|---------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| $x_1 \& x_2 \& x_3$ | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     |

To obtain a function  $\delta_a(x)$  for a **different** value of  $a$ , we can consider a different formula in which we first ‘negate’ (apply the ‘NOT’ operation to) some of the input bits. For example, Table 1.3 describes the function  $\delta_{010}(x)$  on three-bit strings.

Table 1.3: A truth-table for a boolean formula realising the function  $\delta_{010}$ .

| $x_1, x_2, x_3$               | 0,0,0 | 0,0,1 | 0,1,0 | 0,1,1 | 1,0,0 | 1,0,1 | 1,1,0 | 1,1,1 |
|-------------------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| $\neg x_1 \& x_2 \& \neg x_3$ | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     |

We can do this for any bitstring  $a \in \{0,1\}^m$  of any length, just by considering different patterns of negating the bits of the input  $x \in \{0,1\}^m$ .

For a general function  $f : \{0,1\}^m \rightarrow \{0,1\}$  which may obtain the value 1 for multiple inputs  $a_1, a_2, \dots \in \{0,1\}^m$ , we can **combine** the functions  $\delta_{a_j} : \{0,1\}^m \rightarrow \{0,1\}$  which evaluate to 1, for each of the strings  $a_j$ . For instance, in Figure 1.2 we illustrate a function  $f : \{0,1\}^m \rightarrow \{0,1\}$  which evaluates to 1 on four different inputs. We can describe the original function  $f(x)$  in terms of these functions  $\delta_{a_j}(x)$ , by saying that  $f(x) = 1$  if, and only if, **some** function  $\delta_{a_j}(x) = 1$ , for this set of specially indicated input values  $a_j$ . This is the same as taking the ‘logical or’ of the results of the functions  $\delta_{a_1}(x)$ ,  $\delta_{a_2}(x)$ , and so forth. So for example, for the function before:

$$f(x) = \underbrace{\delta_{a_1}(x)}_A \vee \underbrace{\delta_{a_2}(x)}_B \vee \underbrace{\delta_{a_3}(x)}_C \vee \underbrace{\delta_{a_4}(x)}_D \quad (1.7)$$

where we label the individual ‘terms’  $\delta_{a_j}(x)$ , which produce the individual bars in the chart of Figure 1.2.

We can more generally describe any function  $f : \{0,1\}^m \rightarrow \{0,1\}$  in terms of the inputs  $a \in \{0,1\}^m$  where  $f(a) = 1$ , and compute  $f(x)$  by computing whether  $x$  is one of those inputs  $a$  for which  $f(a) = 1$ . For example, consider the function  $f : \{0,1\}^3 \rightarrow \{0,1\}$  given by  $f(x) = x_1 \oplus x_2 \oplus x_3$  (see Table 1.4).

Table 1.4: Truth-table for the boolean expression  $x_1 \oplus x_2 \oplus x_3$ .

| $x_1, x_2, x_3$             | 0,0,0 | 0,0,1 | 0,1,0 | 0,1,1 | 1,0,0 | 1,0,1 | 1,1,0 | 1,1,1 |
|-----------------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| $x_1 \oplus x_2 \oplus x_3$ | 0     | 1     | 1     | 0     | 1     | 0     | 0     | 1     |

Figure 1.2: A sketch of a function on boolean strings, which evaluates to 1 for a few inputs. We consider a more general boolean function  $f : \{0, 1\}^n \rightarrow \{0, 1\}$  using a similar bar chart to Figure 1.1. This function evaluates to 1 for a few inputs, and we point of the individual bars on the chart of height 1 to consider how each one separately can be produced in a logical formula for  $f$ .

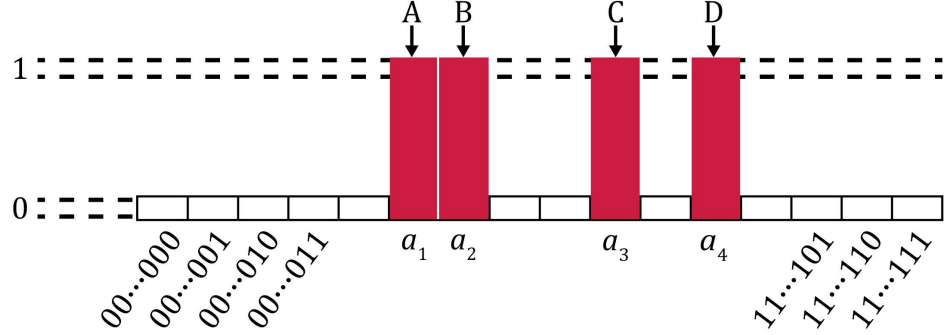


Figure 1.2. Long description: A bar chart. The horizontal axis is labeled by binary strings of unspecified length, starting with  $00 \cdots 000$ ,  $00 \cdots 001$ ,  $00 \cdots 010$ ,  $00 \cdots 011$ , in order of the integers 0, 1, 2, 3 that these would represent. The labeling of the  $x$  axis pauses for several spaces along the axis, and begins again on the right hand side with  $11 \cdots 101$ ,  $11 \cdots 110$ ,  $11 \cdots 111$ , representing the final few  $n$ -bit strings in this ordering. The vertical axis simply has the two boolean values of 0 and 1. In the otherwise unlabelled space on the horizontal axis, there are four positions, labeled by  $a_1$ ,  $a_2$ ,  $a_3$ , and  $a_4$ , meant to represent some generic  $n$ -bit strings. There are bars of height 1 at those indicated positions: the entire rest of the chart has bars of height 0. The bars themselves are given labels to emphasise their importance: the bar at  $a_1$  is labeled by ‘A’, the bar at  $a_2$  is labeled by ‘B’, the bar at  $a_3$  is labeled by ‘C’, and the bar at  $a_4$  is labeled by ‘D’.

Then, we may describe how to compute  $f(x)$  using only the operations ‘ $\neg$ ’, ‘ $\&$ ’, and ‘ $\vee$ ’ by considering the four formulas which describe functions  $\delta_{001}(x)$ ,  $\delta_{010}(x)$ ,  $\delta_{100}(x)$ , and  $\delta_{111}(x)$  which each evaluate to 1 on exactly one input, where  $f(x) = 1$  (see Table 1.5).

Table 1.5: Truth-table describing how to reproduce the function represented by  $x_1 \oplus x_2 \oplus x_3$ , using only  $\neg$ ,  $\&$ , and  $\vee$  operators.

| $x_1, x_2, x_3$                    | 0,0,0 | 0,0,1 | 0,1,0 | 0,1,1 | 1,0,0 | 1,0,1 | 1,1,0 | 1,1,1 |
|------------------------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| $A := \neg x_1 \& \neg x_2 \& x_3$ | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     |
| $B := \neg x_1 \& x_2 \& \neg x_3$ | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     |
| $C := x_1 \& \neg x_2 \& \neg x_3$ | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 0     |
| $D := x_1 \& x_2 \& x_3$           | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     |
| $A \vee B \vee C \vee D$           | 0     | 1     | 1     | 0     | 1     | 0     | 0     | 1     |

(Notice how the final row of this table agrees with the table for  $x_1 \oplus x_2 \oplus x_3$ .)

Figure 1.3: The simple primary-school method to add two numbers, in decimal and in binary. On the left, we have an example of how one might add two numbers, 133 and 29, in the usual decimal notation. When two digits add to 10 or more, a value of 10 is represented by a ‘carry’ of 1 for the next higher value place. On the right, we have an example of how one might add two numbers, 9 and 3, represented in binary here by ‘1001’ and ‘11’. By a similar principle, when the sum of two bits is 2 or more, a value of 2 is represented by a ‘carry bit’ of 1 for the next higher value place.

$$\begin{array}{r}
 \text{decimal:} \quad \begin{array}{r}
 \phantom{0}1 \\
 133 \\
 + 029 \\
 \hline
 162
 \end{array}
 \qquad
 \text{binary:} \quad \begin{array}{r}
 \phantom{0}1\phantom{0}1 \\
 1001 \\
 + 0011 \\
 \hline
 1100
 \end{array}
 \end{array}$$

Figure 1.3. Long description: Two example sums, presented on the left and on the right. In each case, the two numbers to be added are arranged in rows with their digits aligned. Also in each case, the second number to be added happens to involve fewer digits: extra 0 digits are added to the front to represent the two numbers with equal numbers of digits. In the case of the right-hand sum, the numbers happen to be represented in binary.

This same principle applies to any boolean function  $f : \{0, 1\}^m \rightarrow \{0, 1\}$ : in this way, we can compute any such function using only the operators ‘ $\neg$ ’, ‘ $\&$ ’, and ‘ $\vee$ ’. Practically speaking, we almost never describe any meaningful computation by describing how it can be broken down into boolean logical operations acting on an input encoded in binary. In this module, however, this is useful and important when considering how to compute the same functions we’re used to, when working in models of computation that work differently to what we are used to.

### 1.1.5 Using boolean logic to represent arithmetic

What we’ve described above, **can** in principle be used to compute any function on finite sets. The way that we would compute something in practice would generally depend on what it is we want to compute. An important example is how we might perform arithmetic on numbers in a finite range  $0 \leq N < 2^n$ , represented in binary.

To see how to do this, consider how you might add numbers of two or more, using decimal notation (using digits 0–9), by adding individual digits and ‘carrying’ any contribution of 10 to the next digits. When adding numbers in binary, we do **exactly** the same thing – only instead of a one’s place, a ten’s place, a hundred’s place and so forth, we have a one’s place, a two’s place, a four’s place and so forth.

We consider example sums, in decimal and in binary, in Figure 1.3. To add 3 to 9 in binary, we would write 3 in binary as 11 (two plus one), we would write 9 in binary as 1001 (eight plus one). Because three is only a two-bit number, we introduce leading 0 digits to extend it to four bits. Adding together the ‘ones’ digits in the binary example,

we have  $1+1 = 10$ : we then carry the leading 1 from the ones place to the twos place. Adding this into the ‘twos’ digits, we have  $1+0+1 = 10$ : we again carry the leading 1, from the twos place to the fours place. Continuing in this way, we obtain the result 1100 (eight plus four, which is twelve).

The individual steps for performing addition in binary can be represented by simple logical operations. For  $x, y \in \{0, 1\}$ , we have

$$\begin{array}{r} x \\ + y \\ \hline c \ s \end{array} \quad (1.8)$$

where  $s = 1$  if, and only if, either  $x = 1$  and  $y = 0$ , or  $x = 0$  or  $y = 1$ ; and  $c = 1$  if, and only if, both  $x = 1$  and  $y = 1$ . In short: we have  $s = x \oplus y$  and  $c = x \& y$ . If we wished to add two-bit numbers  $x, y \in \{0, 1\}^2$ , we would extend this to:

$$\begin{array}{r} \phantom{c_1} \\ \phantom{+} \phantom{y_1} \phantom{y_0} \\ + \phantom{c_1} \phantom{y_1} \phantom{y_0} \\ \hline c_2 \ s_1 \ s_0 \end{array} \quad \text{for} \quad \begin{cases} s_0 = x_0 \oplus y_0, & c_1 = x_0 \& y_0, \\ s_1 = x_1 \oplus y_1 \oplus c_1, & c_2 = (x_1 \& y_1) \vee (x_1 \& c_1) \vee (y_1 \& c_1); \end{cases} \quad (1.9)$$

here, the formula for  $c_2$  simply comes from considering when the sum  $c_1 + x_1 + y_1$  will be greater than or equal to 2. We can extend this pattern of logical operations, to add numbers which are as large as we want: we apply the same basic principle to compute each (binary) digit of the sum, repeating it as necessary to compute the result.

We can describe similar procedures for subtraction and multiplication. Knowing precisely how to do this will not be important for us in this module: the important thing is to understand the fact that arithmetic **can** be done in this way. (In practice, computers perform arithmetic in registers of fixed size, rather than computing all of the bits required to produce the correct result. This has an interesting impact on the results of those computations, which can be understood in terms of **modular arithmetic**. This is not important to know at the moment, but you will be able to find out more in the note on ‘Binary addition and modular arithmetic’ in the Independent study.

## 1.2 Reversible computation

The way in which we usually describe computation involves over-writing values which were previously stored in variables (for example, to keep track of the maximum value of a list of inputs). However, in order to describe how we process information in quantum computation, it will be useful for us to consider how to perform computation **reversibly**: without any information loss and in a way which we can easily undo.

**Introducing: RAAL.** To describe reversible computation – and to build towards the way in which we will describe quantum computation – we will describe reversible computation using a (fictional) programming language called **RAAL** (pronounced ‘rahl’, to rhyme with ‘call’), for ‘Reversible Abstract Assembly Language’. In **RAAL**, we may allocate ‘registers’ (which you can think of as physical systems to store one or more bits of data): and all registers apart from the ones that store the input, are initialised to 0. But after that,



each individual operation on those registers corresponds to an operation that is performed on the registers, and can be easily reversed. A program in RAAL has a structure of the following form:

- a list of declarations (including declarations of subroutines, of input registers and of extra workspace registers)
- a list of instructions, describing to the input and workspace registers the operations to perform, ending with the instruction ‘**stop**’
- some final declarations to indicate what registers to produce as the output.

We will introduce features of the language a little at a time, as we discuss reversible computation more generally.

### 1.2.1 Storing data, and declaring inputs/outputs

With boolean logic, we describe computations using the boolean operators ‘ $\neg$ ’, ‘ $\&$ ’ and ‘ $\vee$ ’. We think of these as mathematical operators which combined 0s and 1s to produce ‘new’ truth-values – without thinking too much about where these truth-values are stored. We refer to  $m$ -bit strings when considering arithmetic using binary representation, but we don’t consider whether these bits are ‘stored’ some place. But data – even data without any context – does not exist in a vacuum: it has to be stored **somewhere**, as a detectable physical property of some system.

The most basic concept in RAAL is the registers in which data are stored. As we mention above, you should think of a register as a collection of ‘physical bits’, which is to say, the physical system where individual bits of information are stored. In particular, a register is a different thing from the value which is **stored** in the register. In RAAL, we assign names to these registers for the sake of convenience: they then act a little bit like variables. We declare a ‘register’ **a** of a single bit (in other words, a named memory element which can store only the value 0 or 1) by writing

```
bit a
```

and we may declare a register **A** of some number of bits  $m > 0$ , which we may address individually as **A**[0], **A**[1], **A**[2], ..., **A**[ $m-1$ ] as in Java or Python, by writing

```
register A: m bits
```

In some cases we might wish to indicate that a register will come with a pre-loaded input value. To indicate this, we state that it is an input register when we declare the register:

```
input register C: n bits  # stores a string  $x \in \{0,1\}^n$  from the input
input bit t              # stores a bit  $y \in \{0,1\}$  from the input
```

(We allow comments anywhere in RAAL programs, starting with a ‘#’ and continuing to the end of the line.) For all other registers, each bit is considered to be initialised to 0, exactly once, at the very beginning of the computation.

Once we have declared a register, we may also declare at the end of a RAAL program that the value stored in a given register (or individual bits from a register) is to be produced as the output. We can do this using syntax of the form

```
output bit t
output register A
output bit C[0]
```

which, in this case, would produce the three outputs  $t$ ,  $A$  and  $C[0]$  as a sequence (in that order).

In the following, we make a distinction between registers  $a$ ,  $C$ , and so forth, and the values  $x \in \{0, 1\}$ ,  $y \in \{0, 1\}^n$ , and so on, that those registers store. It will be important to describe a distinction between the operations that we perform on registers, and the effect on the values stored in those registers. This will become more important to make once we begin considering quantum computations.

### 1.2.2 Computing basic logical operations

In RAAL we define a very small number of instructions, which will turn out to be all that we need to do practical computation. Specifically, they will allow us to compute each of the boolean logical operations which we considered above.

Each primitive instruction in RAAL acts on either one, two or three bits. These instructions are:

```
NOT    bit1                # an operation with a single operand
CNOT   bit1 , bit2          # an operation with two operands
CCNOT  bit1 , bit2 , bit3 # an operation with three operands
```

We describe each of these operations below.

#### (a) The NOT operation.

We consider an operation NOT as a basic instruction in RAAL, taking a single argument and changing it to its logical negation:

```
NOT a                # perform the map  $x \mapsto \neg x$  on the value stored in ‘a’
```

Figure 1.4 demonstrates a very simple RAAL program, which takes a single bit as input, applies the NOT operation to it, and produces it as output. We can describe the effect of this procedure for both possible values of the input bit  $a$ , using what we will call a **program trace**, as follows. We follow the value of the register  $a$ , at different steps in time, measured by a ‘clock counter’  $t$ . We start at  $t = 0$ , before any instructions have

Figure 1.4: The RAAL program ‘NOT’. A very simple RAAL program, which takes a single bit of data as input (stored in a bit register **a**), negates it, and produces the new value at the output.

```

input bit a      # stores a bit  $x \in \{0,1\}$ 
NOT a           # perform map  $x \mapsto \neg x$  on the value stored in ‘a’
stop
output bit a     # now stores the bit  $\neg x$ 

```

been performed; at time  $t = 1$ , the first (and in this case, only) instruction has been performed. At  $t = 0$ , we consider the possible values that **a** could have, and we consider how that value changes at  $t = 1$  as a result of the NOT operation.

$$\begin{array}{c}
 \boxed{\begin{array}{cc} \text{a : at } t = 0 & 0 \quad 1 \end{array}} \\
 \downarrow \text{NOT a} \\
 \boxed{\begin{array}{cc} \text{a : at } t = 1 & 1 \quad 0 \end{array}}
 \end{array} \tag{1.10}$$

The NOT operation is an allowed operation in RAAL because the map ‘ $x \mapsto \neg x$ ’ is reversible. In particular, it is its own inverse. In boolean logic, ‘double-negation’ has no effect:  $\neg\neg x \equiv x$ . In other words, the map  $x \mapsto \neg x$  is **self-inverse**: if  $\text{NOT} : \{0,1\} \rightarrow \{0,1\}$  is given by  $\text{NOT}(x) = \neg x = 1 - x$ , we have

$$\text{NOT} \cdot \text{NOT} = \text{id}, \tag{1.11}$$

where  $\text{id} : \{0,1\} \rightarrow \{0,1\}$  is the identity map  $\text{id}(x) = x$ . (In fact, we will use the label ‘id’ for identity maps for any size of data: we hope that what we mean will always be clear from the context.) Here, we let  $f \cdot g$  represent the **composition** of two functions,

$$(f \cdot g)(x) = f(g(x)). \tag{1.12}$$

All of this is to say that to undo the effect of the instruction ‘NOT a’, we can simply perform the same instruction ‘NOT a’ again.

### (b) The CNOT operation.

In the above example, once we have performed the operation NOT on the bit **a**, the original value  $x \in \{0,1\}$  of **a** is not immediately available – because it is not stored anywhere that we know of. We could recover it simply by undoing the NOT operation; or alternatively we could consider making a copy of the value of **a** in another register. But simply assigning the value of **a** to another bit is not a reversible operation.

We don’t often think of **copying** as an operation that you have to do, let alone perform reversibly. In programming, we **do** copy data – stored in memory, represented by a variable – but by **assigning it** to other variables, which is an irreversible operation. To **reversibly** make copies of a bit **a**, we need an operation which does this a little more

Figure 1.5: The RAAL program ‘COPY’. A very simple RAAL program, which takes a single bit of data as input (stored in a bit register *a*), adds its value modulo 2 into a newly initialised bit register, and produces both bits as output.

```

input bit a      # stores a bit  $x \in \{0,1\}$ 
bit b            # a fresh bit, initialised to 0
CNOT a, b        # perform the transformation  $0 \mapsto 0 \oplus x$  on ‘b’
stop
output bit a, b  # both store the value  $x$ 

```

gracefully, by **reversibly transforming** the value of  $y$  by the value  $x \in \{0,1\}$  that is stored in *a*. We may do this by computing the ‘XOR’ of  $x$  and  $y$ :

$$y \longmapsto y \oplus x. \quad (1.13)$$

This is the same as adding  $x$  to  $y$  modulo 2: a sort of integer addition where we only keep track of whether the result is even (0) or odd (1). Because ‘even + even = even’ ( $0 \oplus 0 = 0$ ) and ‘odd + odd = even’ ( $1 \oplus 1 = 0$ ), we can reverse this by applying the same ‘addition modulo 2’ operation again. In fact, this is the same as **leaving  $y$  alone** (‘applying’ the identity map  $\text{id}$ ) if  $x = 0$ , and **negating  $y$**  if  $x = 1$ .

To allow us to realise this idea, we introduce an operation **CNOT** as a basic instruction in RAAL, to compute the following function of two bits:

$$\text{CNOT}(x, y) = (x, y \oplus x) = \begin{cases} (x, y), & \text{if } x = 0 \\ (x, \neg y), & \text{if } x = 1 \end{cases} = \begin{cases} (x, x), & \text{if } y = 0 \\ (x, \neg x), & \text{if } y = 1 \end{cases}. \quad (1.14)$$

Using the left-most equality, it is not difficult to show that **CNOT** is self-inverse, as  $y \oplus x \oplus x = y \oplus 0 = y$ :

$$\text{CNOT} \cdot \text{CNOT} = \text{id}. \quad (1.15)$$

Then we may make use of this operation in RAAL, which we realise as an instruction taking two registers as operands:

```

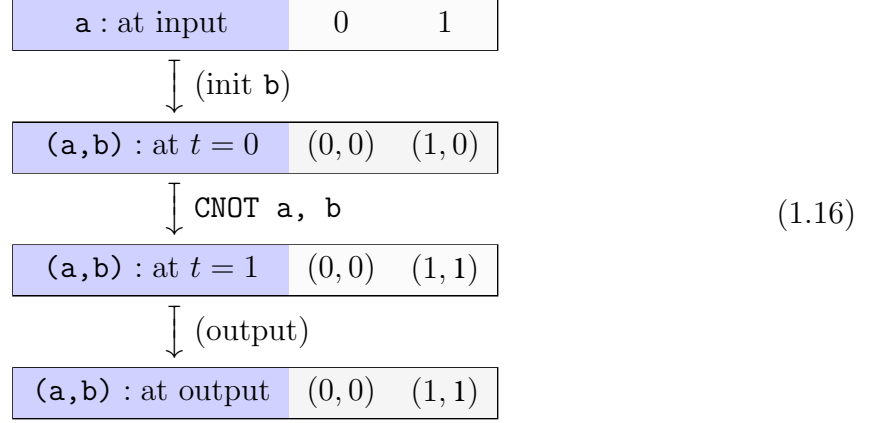
CNOT a, b      # perform transformation  $(x, y) \mapsto (x, y \oplus x)$  on ‘a’ and ‘b’

```

In Eqn. (1.14), the two expressions in curly brackets provide two different perspectives on the **CNOT** operation. The first shows the reason for its name: it describes an operation of negating the value of  $y$ , depending on whether  $x = 1$ . The second one describes how we can use this to copy information: it either copies  $x$  into the second entry (if  $y = 0$ ), or it makes a copy of  $\neg x$  in the second entry (if  $y = 1$ ).

As we suggest above, we can think of **CNOT** as a way of copying information, with a possible modification depending on the value of  $y$ : and if  $y = 0$ , there is no modification made. Figure 1.5 demonstrates a simple RAAL program to illustrate this concept. We can again describe the effect of this procedure, for both possible values of the input *a*,

using a program trace. We extend our previous notion of a ‘program trace’ by explicitly considering what values were provided at the input of the program, and what values are provided at the output. At first, we consider only the input values of **a**, but at time  $t = 0$  we also account for the additional register **b** which is allocated and initialised. We also explicitly note how both **a** and **b** are produced as output.



In this way, we can see explicitly that the effect of the ‘COPY’ program of Figure 1.5 is to make a copy of the input bit.

### (c) The CCNOT operation.

To perform a meaningful computation, we also need some way to realise a boolean logical operation on two inputs. But there is a possible obstacle. For instance, the logical ‘AND’ operation  $\& : \{0, 1\} \times \{0, 1\} \rightarrow \{0, 1\}$  maps two bits to one bit – and so on its own it cannot possibly be reversible, because there is less information at the output than at the input. (In particular, because  $0 \& 0 = 0 \& 1 = 1 \& 0 = 0$ , it is impossible to infer from  $x \& y = 0$  what the specific values of  $x$  and  $y$  are.)

As with copying, we can instead use the result of a logical ‘AND’ operation to **modify** the value of some other variable, using a map

$$z \mapsto z \oplus (x \& y) \tag{1.17}$$

which (reversibly) adds the result of  $x \& y$  to whatever the value of  $z$  is, modulo 2. To allow us to realise this idea, we introduce an operation **CCNOT** as a basic instruction in **RAAL**, to compute the following function of three bits:

$$\text{CCNOT}(x, y, z) = (x, y, z \oplus (x \& y)) = \begin{cases} (x, y, z), & \text{if } x = 0 \text{ or if } y = 0; \\ (x, y, \neg z), & \text{if } x = 1 \text{ and } y = 1. \end{cases} \tag{1.18}$$

We call this a **controlled-controlled-not** (or doubly-controlled-not) operation, because it performs a logical negation on the third bit depending on whether the first two ‘control’ bits are equal to 1. It is also frequently called the ‘Toffoli’ operation in quantum

Figure 1.6: The RAAL program ‘AND’. A very simple RAAL program, which takes two bits of data as input (stored in bit registers *a* and *b*), adds the logical ‘AND’ of their values (using a CCNOT operation) into a newly initialised bit register *r*, and produces *r* as output.

```

input bit a      # stores a bit  $x \in \{0, 1\}$ 
input bit b      # stores a bit  $y \in \{0, 1\}$ 
bit r            # a fresh bit, storing the value 0
CCNOT a, b, r    # perform the transformation  $0 \mapsto 0 \oplus (x \& y)$  on ‘r’
stop
output bit r     # stores the bit-value  $x \& y$ 

```

computation and the theory of reversible computation [1,2]. Again, it is not difficult to show that CCNOT is self-inverse:

$$\text{CCNOT} \cdot \text{CCNOT} = \text{id}. \quad (1.19)$$

Then we may make use of this operation in RAAL, which we realise as an instruction taking three registers as operands:

CCNOT *a*, *b*, *c*      # perform transformation  $(x, y, z) \mapsto (x, y, z \oplus (x \& y))$ .

Again, as we suggest above, we can think of CCNOT as a way of realising a logical ‘AND’ operation. Figure 1.6 demonstrates a simple RAAL program to illustrate this concept. We can again describe the effect of this procedure, for all possible values of the input bits *a* and *b*, using a program trace. We again explicitly consider what values are provided at the input of the program, and what values are provided at the output. At first, we consider only the input values of *a* and *b*, but at time  $t = 0$  we also account for the additional register *r* which is allocated and initialised. At the output, we only describe the value of the bit register *r* which is presented as output.

|                      |         |         |         |         |
|----------------------|---------|---------|---------|---------|
| (a,b) : at input     | (0,0)   | (0,1)   | (1,0)   | (1,1)   |
| ↓ (init r)           |         |         |         |         |
| (a,b,r) : at $t = 0$ | (0,0,0) | (0,1,0) | (1,0,0) | (1,1,0) |
| ↓ CCNOT a, b, r      |         |         |         |         |
| (a,b,r) : at $t = 1$ | (0,0,0) | (0,1,0) | (1,0,0) | (1,1,1) |
| ↓ (output)           |         |         |         |         |
| <i>r</i> : at output | 0       | 0       | 0       | 1       |

(1.20)

In this way, we can see explicitly that the effect of the ‘AND’ program of Figure 1.6 is to compute the logical ‘AND’ of the two bits which are presented as input.

### 1.2.3 Computing more general functions

We can also use the three instructions NOT, CNOT and CCNOT to perform much more general computations. In particular, these three instructions (together with sufficient workspace) are enough to simulate any computation described using boolean logic.

The way in which we realise copying, and the logical ‘AND’ operation we have described, shows the way that we can **define** a reversible counterpart for any function  $f : \{0, 1\}^m \rightarrow \{0, 1\}$ , which would then allow us to write a RAAL program to compute  $f$ . For any such function  $f$ , we can define a self-inverse function  $P_f : \{0, 1\}^{m+1} \rightarrow \{0, 1\}^{m+1}$  given by

$$P_f(x, r) := (x, r \oplus f(x)) \quad (1.21)$$

for  $x \in \{0, 1\}^m$ . More generally, for  $F : \{0, 1\}^m \rightarrow \{0, 1\}^n$ , we may define a function

$$P_F(x, r) = (x, r \oplus f(x)) \quad (1.22)$$

for  $x \in \{0, 1\}^m$  and  $r \in \{0, 1\}^n$ , where we write  $r \oplus s = (r_1 \oplus s_1, r_2 \oplus s_2, \dots, r_n \oplus s_n)$  for two  $n$ -bit strings  $r$  and  $s$ . By defining functions such as  $P_f$ , we may compute a function  $f$  reversibly, by computing

$$P_f(x, 0) = (x, f(x)) \quad (1.23)$$

and then just using the register storing  $f(x)$  as the output. If we can initialise the bits of the output registers to 0, we then only need to find a way to compute the function  $P_f$ . To describe how to actually compute  $P_f$  for a given function  $f : \{0, 1\}^m \rightarrow \{0, 1\}$ , it will be useful to describe how to compute other functions as subroutines. In RAAL, we define the following syntax to define a named subroutine, which can act on any number of registers as ‘operands’:

```
define SUBRNAME reg1 , reg2 , ... , regn :
  # 1. declare the types of the operands
  # 2. declare any local registers used as workspace
  # 3. list of operations to perform, implementing ‘SUBRNAME’
  # 4. a final instruction, ‘stop’
stop
```

The ‘operands’ are registers whose values will generally be affected by the subroutine, and so are both the input and the output. (The technical description is that these are procedures with ‘side-effects’.) Any ‘local’ registers which are defined as a part of the subroutine, are accessible only to the instructions described in the subroutine. (Two different subroutines SUBR.ONE and SUBR.TWO can each have completely different local registers named ‘w’, without sharing them.) For ease of reading, we adopt the convention of emphasising the scope of such declarations using indentation as in Python. But it is the declaration **stop** which indicates the end of the subroutine: and as we do not need to specify a separate output, that defines the end of the declaration of the subroutine.

The operations in a subroutine can consist of the instructions NOT, CNOT, and CCNOT, but can also consist of other subroutines which are defined separately (earlier in the program

or in a separate file). In this way, we can build more complex computations in RAAL from simpler ones, in the way you might expect to do with a computer program or when solving a problem more generally. We will consider some examples as follows.

**(a) Computing ‘OR’ on two bits.**

We may reversibly compute the logical ‘OR’ of two bits by making use of the logical equivalence  $x \vee y \equiv \neg(\neg x \& \neg y)$ , which allows us to compute the logical ‘OR’ using the logical ‘NOT’ and logical ‘AND’. This motivates the RAAL subroutine demonstrated in Figure 1.7, for a reversible operation P\_OR2. This computes the function  $P_{\text{OR}_2}(x, y, z) =$

Figure 1.7: The RAAL procedure ‘P\_OR2’. This procedure takes three single-bit registers a, b and c, and computes the logical ‘OR’ of the values of a and b ‘into’ the value of the third register c (by adding modulo 2 to the value of the c).

```
define P_OR2 a, b, c :
  # compute the logical ‘OR’ of ‘a’ and ‘b’ into ‘c’
  operand bit a, b, c # declare each operand to be a single bit

  NOT a           # perform the map  $x \mapsto \neg x$  on value in ‘a’
  NOT b           # perform the map  $y \mapsto \neg y$  on value in ‘b’
  CCNOT a, b, c   # compute  $z \oplus (\neg x \& \neg y)$  in ‘c’
  NOT c           # compute  $z \oplus (\neg x \& \neg y) \oplus 1 \equiv z \oplus (x \vee y)$  in ‘c’

  NOT a           # undo the logical negation on ‘a’
  NOT b           # undo the logical negation on ‘b’
  stop
```

$(x, y, z \oplus (x \vee y))$ , which is the reversible counterpart of the function  $\text{OR}_2(x, y) = x \vee y$ . We may see this by tracing the effects of the instructions, in sequence, for each value of a and b (and considering just the case  $c = 0$  in which we expect to compute the logical ‘OR’ of the values of a and b). In some cases, we describe the effect of two operations at once, where we can do this without confusion: when we do this, we may skip over some time-steps (the value of  $t$  will increase by more than 1).



|                      |                            |           |           |           |        |
|----------------------|----------------------------|-----------|-----------|-----------|--------|
| (a,b,c) : at $t = 0$ | (0, 0, 0)                  | (0, 1, 0) | (1, 0, 0) | (1, 1, 0) |        |
|                      | $\downarrow$ NOT a ; NOT b |           |           |           |        |
| (a,b,c) : at $t = 2$ | (1, 1, 0)                  | (1, 0, 0) | (0, 1, 0) | (0, 0, 0) |        |
|                      | $\downarrow$ CCNOT a, b, c |           |           |           |        |
| (a,b,c) : at $t = 3$ | (1, 1, 1)                  | (1, 0, 0) | (0, 1, 0) | (0, 0, 0) | (1.24) |
|                      | $\downarrow$ NOT c         |           |           |           |        |
| (a,b,c) : at $t = 4$ | (1, 1, 0)                  | (1, 0, 1) | (0, 1, 1) | (0, 0, 1) |        |
|                      | $\downarrow$ NOT a ; NOT b |           |           |           |        |
| (a,b,c) : at $t = 6$ | (0, 0, 0)                  | (0, 1, 1) | (1, 0, 1) | (1, 1, 1) |        |

The final two operations, ‘NOT a’ and ‘NOT b’, are meant to undo the effect of the earlier ‘NOT a’ and ‘NOT b’ instructions – restoring a and b to their original input values. This is a special example of something known as ‘uncomputation’, which allows us to make use of the same pieces of memory without having to make many copies of the same data. By restoring the values of a and b to what they originally were, we make it easier to use those values again (essentially, performing a small act of good software engineering).

### (b) Subroutines for other two-bit logical operators.

To help us to describe some things a bit more uniformly, we can define reversible procedures of this sort for the other logical operators ‘ $\oplus$ ’ and ‘ $\&$ ’ as well. If we define the function  $\text{XOR}_2(x, y) = x \oplus y$ , it follows that  $P_{\text{XOR}_2}(x, y, z) = (x, y, z \oplus (x \oplus y))$ , which we may easily realise by the procedure P\_XOR2 demonstrated in Figure 1.8. Also shown is an implementation of a procedure P\_AND2, which computes the function  $P_{\text{AND}_2}$  which is the reversible counterpart of the function  $\text{AND}_2(x, y) = x \& y$ . In the latter case, this may seem a bit redundant, as it does nothing more than the instruction CCNOT does: we define it in order to allow ourselves to consistently refer to P\_OR2, P\_XOR2, and P\_AND2 as ways to reversibly realise the boolean logical ‘OR’, ‘XOR’ and ‘AND’ operations.

### (c) Subroutines with local registers, and uncomputation.

You may have wondered whether RAAL should also include more versions of the CNOT and CCNOT instruction, such as a ‘controlled-controlled-controlled-not’ operation CCCNOT:

$$\text{CCCNOT}(w, x, y, z) = \begin{cases} (w, x, y, z), & \text{if } w = 0, x = 0, \text{ or } y = 0; \\ (w, x, y, \neg z), & \text{if } w = x = y = 1. \end{cases} \quad (1.25)$$

While we can define such a function, there is no need to include this as a basic operation in RAAL in order to be able to perform it: we can just recognise that

$$\text{CCCNOT}(x, y, z, t) = (x, y, z, t \oplus (x \& y \& z)) = P_{\text{AND}_3}(x, y, z, t), \quad (1.26)$$

Figure 1.8: The RAAL procedures ‘P\_XOR2’ and ‘P\_AND2’. These procedures take three single-bit registers a, b and c, and compute (respectively) the logical ‘XOR’ of the values of a and b, or the logical ‘AND’ of the values of a and b, into the value of the third register c.

```
define P_XOR2 a, b, c :
  # compute the logical ‘XOR’ of ‘a’ and ‘b’ into ‘c’
  operand bit a, b, c # declare each operands to be a single bit
  CNOT a, c          # add the value in ‘a’ modulo 2, to ‘c’
  CNOT b, c          # add the value in ‘b’ modulo 2, to ‘c’
  stop

define P_AND2 a, b, c :
  # compute the logical ‘AND’ of ‘a’ and ‘b’ into ‘c’
  # ... which is to say, perform a CCNOT operation
  operand bit a, b, c
  CCNOT a, b, c
  stop
```

Figure 1.9: The RAAL procedure ‘P\_AND3’. This procedure takes four single-bit registers a, b, c and d, and computes the logical ‘AND’ of the values of a, b and c into the value of the fourth register d.

```
define P_AND3 a, b, c, d :
  # compute the logical ‘AND’ of ‘a’, ‘b’, ‘c’, into ‘d’
  operand bit a, b, c, d # declare the operands to be single bits
  bit w                  # extra bit of working memory, initially 0

  P_AND2 a, b, w          # compute logical ‘AND’ of ‘a’ and ‘b’ into ‘w’
  P_AND2 c, w, d          # compute logical ‘AND’ of ‘c’ and ‘w’ into ‘d’
  P_AND2 a, b, w          # UN-compute the value of ‘a & b’
  stop
```

where  $\text{AND}_3(x, y, z) = x \& y \& z$ . With some additional workspace, we can compute  $x \& y$  in a local single bit register, and then compute  $(x \& y) \& z$  into a target bit register. We could realise this as a subroutine to perform  $P_{\text{AND}_3}$  as demonstrated in Figure 1.9.

Notice the final operation of P\_AND3: this clears away the partial working stored in w, restoring its original value of 0. This is another example of **uncomputation**. We saw this before with the definition of P\_OR2 in Figure 1.7, when we performed extra operations to restore the values of the input operands. In reversible computation, the purpose of ‘uncomputation’ is to undo the effect of operations that we have performed, to restore some or all of the data registers to some earlier value (such as an initial value of 0 for work registers, or the original value of input registers). The reason for this, is that the register w is only initialised one time: at the beginning of the program – and not at the

Figure 1.10: The RAAL procedures ‘P\_OR3’ and ‘P\_XOR3’. These procedures take four single-bit registers a, b, c and d, and compute (respectively) the logical ‘OR’ or the logical ‘XOR’ of the values of a, b and c, into the value of the fourth register d.

```

define P_OR3 a, b, c, d :
  # compute the logical ‘OR’ of ‘a’, ‘b’, ‘c’, into ‘d’
  operand bit a, b, c, d # declare the operands to be single bits
  bit w                  # extra bit of working memory, initially 0

  P_OR2 a, b, w          # compute logical ‘OR’ of ‘a’ and ‘b’ into ‘w’
  P_OR2 c, w, d          # compute logical ‘OR’ of ‘c’ and ‘w’ into ‘d’
  P_OR2 a, b, w          # UN-compute the value of ‘a ∨ b’
  stop

define P_XOR3 a, b, c, d :
  # compute the logical ‘XOR’ of ‘a’, ‘b’, ‘c’, into ‘d’
  operand bit a, b, c, d # declare the operands to be single bits
  CNOT a, d
  CNOT b, d
  CNOT c, d
  stop

```

beginning of the subroutine. (This may seem unusual to some students, but it is similar to how member variables behave for persistent objects in object oriented programming.) To ensure that the procedure works properly each time it is used, we should ensure that **w** has an initial value of 0 for every invocation of **P\_AND3**, by reversibly restoring the value of **w** at the end of the procedure.

For the sake of completeness, we define subroutines **P\_OR3** and **P\_XOR3** to compute the logical ‘OR’ and ‘XOR’ of three bits. These are demonstrated in Figure 1.10. (Notice how in the case of **P\_XOR3**, we don’t actually need any additional workspace.)

### 1.2.4 Universality

Using just the instructions **NOT**, **CNOT**, and **CCNOT** we can reversibly compute any function  $\{0, 1\}^m \rightarrow \{0, 1\}$ , and by extension any function  $\{0, 1\}^m \rightarrow \{0, 1\}^n$ , for any  $m, n > 0$ . To do so, we only need to use the ideas that we saw for the universality of (‘non-reversible’) boolean logic, and allow ourselves enough extra bits as workspace. Furthermore, we can simplify this task by building more complex subroutines from these instructions, and then using these subroutines to define still more complex subroutines.

As an example, we may consider how to use the subroutines above to compute the sum of two integers  $X, Y \in \{0, 1, 2, 3\}$ , expressed in binary by bitstrings  $x_1x_0, y_1y_0 \in \{0, 1\}^2$ . The result will be an integer  $Z = X + Y \in \{0, 1, \dots, 6\}$ , which may be represented by a

three-bitstring  $z_2 z_1 z_0 \in \{0, 1\}^3$ . This defines a function  $\text{ADD}_2 : \{0, 1\}^2 \times \{0, 1\}^2 \rightarrow \{0, 1\}^3$ , whose reversible counterpart  $P_{\text{ADD}_2}$  we may define by

$$P_{\text{ADD}_2}(x, y, r) = (x, y, r \oplus \text{ADD}_2(x, y)) = (x, y, r \oplus (x+y)) \quad (1.27)$$

where again  $r \oplus (x+y)$  is a bit-wise ‘XOR’ of the sum  $x+y$  with the string  $r$ . We may use the logical formulae of Eqn. (1.9) to describe how the bits of  $z$  depend on the bits of  $x$  and  $y$  (defining  $c_1 = x_0 \& y_0$  for the carry bit from the units place):

$$z_0 = x_0 \oplus y_0, \quad z_1 = x_1 \oplus y_1 \oplus c_1, \quad z_2 = (x_1 \& y_1) \vee (x_1 \& c_1) \vee (y_1 \& c_1). \quad (1.28)$$

Our procedure will then need working memory to compute the sub-expressions

$$c_1 = x_0 \& y_0, \quad w_0 = x_1 \& y_1, \quad w_1 = x_1 \& c_1, \quad w_2 = y_1 \& c_1. \quad (1.29)$$

We then proceed by defining a procedure `P_ADD2`, as demonstrated in Figure 1.11. It takes 2-bit registers `a` and `b` as input as well as a 3-bit target register `s`, declares enough auxiliary working memory to store the partial results  $c_1, w_0, w_1$ , and  $w_2$ , systematically computes each expression that we need using the appropriate logical operation, and uncomputes the partial workings to restore the auxiliary memory bits to the value of 0. Notice that the ‘uncomputation’ operations are performed in the reverse order from the original computation of the partial workings: in effect we perform the uncomputation by ‘rewinding’ the original computation, undoing each operation one at a time until they have each been undone. Each computational step can be expressed as a logical operation (such as `P_AND2`, which is really just an alias for `CCNOT`) or a subroutine with its own memory requirements already accounted for (such as `P_OR3`).

### 1.2.5 Reflection

This approach to reversible computation generalises to all computations that can be described using boolean logic. Using the reversible instructions / subroutines `NOT`, `P_AND2`, `P_OR2`, and `P_XOR2` to compute the outcomes of boolean logical operators, we can reversibly compute any function step-by-step from a description of how to compute a function  $f : \{0, 1\}^m \rightarrow \{0, 1\}^n$ , as follows:

1. Determine all of the partial results that need to be computed (and their dependency on one another).
2. In a `RAAL` program, declare enough auxiliary memory registers as workspace to store the partial results (and/or output registers to store the final results).
3. In that same `RAAL` program, use `NOT`, `CNOT` and `CCNOT` instructions (and subroutines, as desired) to compute those results.
4. Have the `RAAL` program produce the output, or – if we are defining a subroutine – uncompute all of the partial workings in the auxiliary memory.

We could even consider questions of how to do such reversible computations more efficiently, either in terms of reducing the number of operations or the amount of auxiliary memory. This will not be essential for us, but such questions are important to address in practice when we apply these same ideas to quantum hardware.

Figure 1.11: The RAAL procedure ‘P\_ADD2’. This procedure takes two 2-bit registers *a* and *b*, and a 3-bit register *s*, and computes the sum of *a* and *b* into *s* (by a bit-wise ‘XOR’ of the result into the bits of *s*).

```

define P_ADD2 a, b, s :
  # compute the sum of the values stored in ‘a’ and ‘b’, into ‘s’;
  # where ‘a’, ‘b’ are 2-bit registers, and ‘s’ is a 3-bit register
  operand register a, b: 2 bits
  operand register s: 3 bits
  bit c1                                # workspace for unit-place carry bit
  register w: 3 bits                    # workspace for twos-place carry bit

  P_XOR2 a[0], b[0], s[0]              # compute units place of the sum
  P_AND2 a[0], b[0], c1                # compute unit-place carry bit

  P_AND2 a[1], b[1], w[0]
  P_AND2 a[1], c1, w[1]
  P_AND2 b[1], c1, w[2]
  # partial workings to compute carry bit from twos place

  P_XOR3 a[1], b[1], c1, s[1]          # compute twos place of the sum
  P_OR3 w[0], w[1], w[2], s[2]         # compute fours place of the sum

  P_AND2 b[1], c1, w[2]
  P_AND2 a[1], c1, w[1]
  P_AND2 a[1], b[1], w[0]
  # uncompute partial workings for carry bit from twos place

  P_AND2 a[0], b[0], c1
  # uncompute carry bit from the units place
  stop

```

### 1.3 Randomised computation

Sometimes it can be useful to deliberately introduce randomness into a computation. It might sound strange to rely on operations which do not produce predictable results. But sometimes it is easier to stumble upon a successful next step in solving a problem, rather than doing something which is systematic and predictable but slower.

Relying on randomness introduces a chance that things will not go as well as you might hope. The question is what risks you are willing to take in the hopes of solving a problem faster. To determine whether this trade-off is worthwhile, it depends on whether you know enough about the problem you are trying to solve to make it likely that you will ‘stumble’ on a useful piece of data, and thereby obtain a measurably faster algorithm. To read more about these issues, see the notes on ‘Asymptotic notation and run-time

complexity’, and on ‘Randomness and primality tests’ in the Independent study.

For now, we will instead consider how to **describe** randomised computation, in a way that allows us to determine all the possible outcomes of the computation.

### 1.3.1 Stepping through randomised algorithms?

We have already seen a notion of ‘program trace’ in the case of some of the RAAL programs above. We could do the same thing for randomised algorithms, if whenever we involve some randomness (such as a coin flip or a die roll), we consider **specific outcomes** of that random process. The procedure would then look much like a deterministic algorithm, apart from the unpredictable values assigned to the random variables. But, this would not allow us to directly consider such questions as the probability with which the algorithm produces any given answer.

There is another approach to stepping through randomised algorithms: to track the probability with which any outcome could occur. This is not an **efficient** way to analyse a randomised algorithm, but it will introduce a way of looking at non-deterministic computation which will prove helpful later when we come to look at quantum computation.

The thing which is common to all randomised computations is that they involve **probability distributions** over the possible, random values that the algorithm involves. The usual way in which one describes probabilities over a finite set  $S$  is by a **probability distribution**, which is a function  $p : S \rightarrow \mathbb{R}$  satisfying the properties

- $0 \leq p(s) \leq 1$  for all  $s \in S$ ;
- $\sum_{s \in S} p(s) = 1$ : for  $S = \{s_0, s_1, \dots, s_{N-1}\}$ , we have  $p(s_0) + p(s_1) + \dots + p(s_{N-1}) = 1$ .

However, it will be more practical for us to instead describe probability distributions (and similar sorts of distributions) by vectors: a one-dimensional array of ‘coefficients’,

$$\mathbf{p} = \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{N-1} \end{bmatrix} \in \mathbb{R}^N, \quad (1.30)$$

where we let  $p_k$  stand for the probability of choosing the element  $s_k \in S$ . Again, we have  $p_k \geq 0$  for each coefficient, and  $p_0 + p_1 + \dots + p_{N-1} = 1$ . For instance, if  $S = \{1, 2, 3, 4, 5, 6\}$  is the set of possible outcomes when rolling a standard six-sided die, we might consider representing the elements of  $S$  by three-bit strings, and then describing the distribution

Figure 1.12: The RAALcr program ‘SHARED\_RANDOMNESS’. This procedure produces two bits, which are individually random but guaranteed to have the same value if **b** is initially 0, or guaranteed to have the opposite value if **b** is initially 1.

```

input bit a, b      # two bits taken as input
!randomise a        # randomise the value stored in 'a'
CNOT a, b
stop
output bit a, b

```

of the die-roll  $\mathbf{d}$  by a distribution over three-bit strings:

$$\mathbf{d} = \begin{bmatrix} d_{000} \\ d_{001} \\ d_{010} \\ d_{011} \\ d_{100} \\ d_{101} \\ d_{110} \\ d_{111} \end{bmatrix} = \begin{bmatrix} 0 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 0 \end{bmatrix} \quad (1.31)$$

where the possible outcomes of the die are represented by the six strings 001, 010, 011, 100, 101, 110 (representing the numbers 1–6, with no chance of rolling a ‘0’ or a ‘7’).

The way that we consider using randomness in this case, is to define a language **RAALcr** (pronounced ‘rollicker’), which is **RAAL** extended with a control system and randomness. We can think of the control system as providing the reversible processor with the instructions to perform, and with the input values and random values that it computes with. The reversible processor in turn provides the control system with the output of the computation, and the information needed for the control flow for the computation.

The first instruction which we introduce in **RAALcr** allows the control system to produce a random bit value in the memory that the reversible computer works on:

```

!randomise a      # change 'a' to be 0/1 with equal probability

```

This is an irreversible operation: whether or not the value of **a** is 0 or 1 to begin with, afterwards it could be 0 or 1 with equal probability, so that the value of **a** beforehand cannot be recovered. We should think of this as an operation involving the control system at run-time: any such operation will be written with a ‘!’ at the beginning.

We consider a notion of a ‘program trace’ for a simple **RAALcr** program, such as the one illustrated in Figure 1.12. In this program, we have two bits, **a** and **b**, which are taken as input. We then randomise **a**, and perform a **CNOT** operation to ‘copy’ the value of **a** into **b**. Suppose that **a** and **b** are both initially 0. The vector representing the probabilities

of the possible values of  $(\mathbf{a}, \mathbf{b})$  then changes as follows as we perform the instructions in that program:

$$\begin{array}{l} \text{initial} \\ \text{distribution:} \end{array} \begin{bmatrix} \Pr[(0,0)] \\ \Pr[(0,1)] \\ \Pr[(1,0)] \\ \Pr[(1,1)] \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \xrightarrow{\text{!randomise a}} \begin{bmatrix} 1/2 \\ 0 \\ 1/2 \\ 0 \end{bmatrix} \xrightarrow{\text{CNOT}} \begin{bmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{bmatrix} \quad (1.32)$$

At first, we have  $\mathbf{a} = 0$  and  $\mathbf{b} = 0$ , so we have  $\Pr[(0,0)] = 1$ , and a probability of zero (impossibility) for the other values. Once we randomise the value of  $\mathbf{a}$ , we see that  $(\mathbf{a}, \mathbf{b})$  taken together can either be  $(0,0)$  or  $(1,0)$ , and these two possibilities each occur with probability  $1/2$ . When we perform the CNOT operation, the memory does not change if  $(\mathbf{a}, \mathbf{b}) = (0,0)$ , but if instead  $(\mathbf{a}, \mathbf{b}) = (1,0)$  before the operation, it changes to  $(1,1)$  after. At the end of the procedure, the two bits  $\mathbf{a}$  and  $\mathbf{b}$  are each randomly 0 or 1, but have the same value as each other.

Two variables or registers  $\mathbf{A}$  and  $\mathbf{B}$  are **independent** of one another, if

$$\Pr[(\mathbf{A}, \mathbf{B}) = (x, y)] = \Pr[\mathbf{A} = x] \cdot \Pr[\mathbf{B} = y], \quad (1.33)$$

which, for example, occurs if their values were generated without involving each other. Notice that the distribution at the end of Eqn. (1.32) is **not** such a distribution: there is no way for  $\mathbf{a}$  to have a distribution vector  $\mathbf{p} \in \mathbb{R}^2$  and for  $\mathbf{b}$  to have an independent distribution vector  $\mathbf{q} \in \mathbb{R}^2$ , such that  $p_0q_0 = p_1q_1 = 1/2$  but  $p_0q_1 = p_1q_0 = 0$ . This makes it important to consider their **joint distribution**: the way in which their random values are distributed when taken together, as we do in Eqn. (1.32).

We could do a similar analysis for each of the possible values of  $\mathbf{a}$  and  $\mathbf{b}$  at the beginning of the procedure. For instance, if initially  $\mathbf{a} = 1$  and  $\mathbf{b} = 1$ , we would instead have the following changes in the probability distribution over the values of  $\mathbf{a}$  and  $\mathbf{b}$ :

$$\begin{array}{l} \text{initial} \\ \text{distribution:} \end{array} \begin{bmatrix} \Pr[(0,0)] \\ \Pr[(0,1)] \\ \Pr[(1,0)] \\ \Pr[(1,1)] \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{\text{!randomise a}} \begin{bmatrix} 0 \\ 1/2 \\ 0 \\ 1/2 \end{bmatrix} \xrightarrow{\text{CNOT}} \begin{bmatrix} 0 \\ 1/2 \\ 1/2 \\ 0 \end{bmatrix} \quad (1.34)$$

We see then, that it is possible to consider stepping through a randomised algorithm. What we will see next is that, in fact, we can describe the way that the probability distribution changes with each instruction, by transforming them using matrix multiplication.

### 1.3.2 Conditional probabilities and computation

We can describe the way that probability distributions change with each instruction in a RAALcr program, using **conditional probabilities**. This is the probability of one thing  $\mathbf{B}$  having some value  $y$ , given that some other thing  $\mathbf{A}$  happens to have some value  $x$ ,



which we can write as  $\Pr[B = s \mid A = r]$ . As it happens, we can consider the situation where **A** and **B** are both the same register(s), but where **A** describes their value before some instruction, and **B** describes their value after the instruction. For instance, we can summarise the effect of the program of Figure 1.12 in terms of the output distribution over  $(\mathbf{a}, \mathbf{b}) = x'y' \in \{0, 1\}^2$ , for each possible definite input value of  $(\mathbf{a}, \mathbf{b}) = xy \in \{0, 1\}^2$ :

$$\begin{array}{llll} \Pr[00 \mid 00] = \frac{1}{2} & \Pr[00 \mid 01] = 0 & \Pr[00 \mid 10] = \frac{1}{2} & \Pr[00 \mid 11] = 0 \\ \Pr[01 \mid 00] = 0 & \Pr[01 \mid 01] = \frac{1}{2} & \Pr[01 \mid 10] = 0 & \Pr[01 \mid 11] = \frac{1}{2} \\ \Pr[10 \mid 00] = 0 & \Pr[10 \mid 01] = \frac{1}{2} & \Pr[10 \mid 10] = 0 & \Pr[10 \mid 11] = \frac{1}{2} \\ \Pr[11 \mid 00] = \frac{1}{2} & \Pr[11 \mid 01] = 0 & \Pr[11 \mid 10] = \frac{1}{2} & \Pr[11 \mid 11] = 0. \end{array} \quad (1.35)$$

We can arrange these into a  $4 \times 4$  array, represented by a matrix  $P$  where the rows are indexed by  $x'y' \in \{0, 1\}^2$  and the columns are indexed by  $xy \in \{0, 1\}^2$ , so that  $P_{x'y', xy} = \Pr[x'y' \mid xy]$ . Then we would write

$$P = \begin{bmatrix} 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 1/2 \\ 0 & 1/2 & 0 & 1/2 \\ 1/2 & 0 & 1/2 & 0 \end{bmatrix}. \quad (1.36)$$

This is useful for more than just presenting information in a condensed way: we can in fact use this array as a matrix  $P : \mathbb{R}^4 \rightarrow \mathbb{R}^4$ , describing the transformation which happens for **any** input distribution of values of  $(\mathbf{a}, \mathbf{b})$  under this procedure. We simply take any distribution  $\mathbf{s}$  over the possible input values of  $(\mathbf{a}, \mathbf{b})$ , and multiply it by  $P$ . For instance, if initially  $\mathbf{a} = 0$  and  $\mathbf{b}$  is uniformly random (a 50/50 distribution over 0 and 1), we have

$$\mathbf{s} = \begin{bmatrix} 1/2 \\ 1/2 \\ 0 \\ 0 \end{bmatrix} \xrightarrow{\text{!randomise a, CNOT}} P\mathbf{s} = \frac{1}{2} \begin{bmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{bmatrix} + \frac{1}{2} \begin{bmatrix} 0 \\ 1/2 \\ 1/2 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}. \quad (1.37)$$

On the other hand, for an input where  $\mathbf{a}$  is uniformly random but  $\mathbf{b}$  is initially 0,

$$\mathbf{s} = \begin{bmatrix} 1/2 \\ 0 \\ 1/2 \\ 0 \end{bmatrix} \xrightarrow{\text{!randomise a, CNOT}} P\mathbf{s} = \frac{1}{2} \begin{bmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{bmatrix} + \frac{1}{2} \begin{bmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 0 \\ 0 \\ 1/2 \end{bmatrix}. \quad (1.38)$$

In general, suppose we have some register **A** which takes values over some finite set  $S = \{s_0, s_1, \dots, s_{N-1}\}$ , and we want to consider the probability that it takes some value  $r \in S$  after some program or operation **OPN** which only acts on **A**. The **law of total probability** from probability theory states that:

$$\Pr[r] = \sum_{s_k \in S} \Pr[r \mid s_k] \cdot \Pr[s_k]. \quad (1.39)$$

This actually represents a matrix multiplication, relating the distribution  $\mathbf{s}$  before OPN (representing the probabilities  $\Pr[s_k]$ ) to the distribution  $\mathbf{r}$  after OPN (representing the probabilities  $\Pr[r]$  of outcomes  $r$ ), using a matrix  $P = [P_{r,s_k}]$  given by the conditional probabilities  $P_{r,s_k} = \Pr[r | s_k]$ . That is, we can condense Eqn. (1.39) to the form

$$\mathbf{r} = P\mathbf{s}. \quad (1.40)$$

In particular: for each  $k \in \{0, 1, \dots, N-1\}$ , let  $\mathbf{p}^{(k)}$  be the probability distribution which we obtain at the output given a definite input value of  $s_k$  (occurring with probability 1). Then the matrix  $P$  is the matrix whose columns are just the probability vectors  $\mathbf{p}^{(k)}$ :

$$P = \begin{bmatrix} \mathbf{p}^{(0)} & \mathbf{p}^{(1)} & \dots & \mathbf{p}^{(k)} & \dots & \mathbf{p}^{(N-1)} \end{bmatrix}. \quad (1.41)$$

### 1.3.3 Operators for basic RAALcr instructions, and ‘ket’ notation

Having seen that we can describe computations on probability distributions by transformations of the probability vectors, we may consider what matrices correspond to the basic instructions of RAALcr: NOT, CNOT, CCNOT and !randomise.

Three of these instructions are in fact deterministic, describing a definite output value for a given input value. This motivates describing an easy way to represent probability distributions, which in fact describe some definite input or output value. To do this, we define a special notation which we will use for these distributions: ‘ket’ notation. In this module, we define ‘ket’ notation to represent a specific vector  $\mathbf{v}$  with coefficients  $v_x$ , for various bitstrings  $x \in \{0, 1\}^n$ . For a bitstring  $x \in \{0, 1\}^n$ , we write

$$|x\rangle \quad (\text{pronounced: ‘ket } x\text{’}) \quad (1.42)$$

to represent a vector  $\mathbf{v} \in \mathbb{R}^{2^n}$  such that  $v_x = 1$ , and  $v_z = 0$  for all other  $z \neq x$ . For instance, for  $x \in \{0, 1\}$ ,

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \quad (1.43)$$

for  $x \in \{0, 1\}^2$ , the vectors  $|x_1x_2\rangle$  are

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |01\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad |10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad |11\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}; \quad (1.44)$$

and for  $x \in \{0, 1\}^3$ , the vectors  $|x_1x_2x_3\rangle$  are

$$|000\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |001\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |010\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |011\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |100\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \text{and so on.} \quad (1.45)$$

The deterministic and reversible operations on definite values, map distributions  $|x\rangle$  representing definite values on one, two or three bits to other such ‘definite’ distributions. This notation (also known as ‘Dirac’ notation) is seen most often when describing quantum computation or quantum physics. But it is just a different style of vector notation – which in this case is helpful to describe probability distributions over different definite values which may be stored in memory.

We will use ‘NOT’ to also refer to the matrix describing how the NOT instruction transforms probability distributions on single bits, and similarly use ‘CNOT’ and ‘CCNOT’ to refer to the matrices which describe how those instructions transform distributions on two- and three-bit strings. For instance, we may write  $\text{NOT}|0\rangle = |1\rangle$  and  $\text{NOT}|1\rangle = |0\rangle$ , as well as  $\text{CNOT}|00\rangle = |00\rangle$ ,  $\text{CNOT}|01\rangle = |01\rangle$ ,  $\text{CNOT}|10\rangle = |11\rangle$ , and  $\text{CNOT}|11\rangle = |10\rangle$ , and so forth. Also, for the sake of brevity, we let ‘ $C$ ’ refer to the effect of the `!randomise` instruction. Then, we can describe the effect of each instruction of `RAALcr` using the matrices shown in Figure 1.13.

Figure 1.13: Matrices describing how the basic instructions of `RAALcr` transform input probability distributions over possible values of registers, to output distributions. (The definitions of the operations NOT, CNOT, and CCNOT are given in Section 1.2.2(a), Section 1.2.2(b), and Section 1.2.2(c) respectively.)

$$\begin{aligned} \text{NOT} &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} & \text{CNOT} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} & \text{CCNOT} &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \\ C &= \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix} \end{aligned}$$

In the case of NOT, CNOT and CCNOT, because the instructions are reversible and deterministic, the corresponding matrices are **permutation operators**: operators  $P$  which have a single 1 in each row, and in each column, with all other coefficients being equal to 0. These matrices are always invertible, and in particular satisfy  $P^{-1} = P^\top$ .

### 1.3.4 Transforming joint distributions

The operators described above transform probability vectors, for **exactly** the number of bits that they operate on. For instance, the NOT operator acts on probability vectors for a single bit: this does not tell us how the NOT instruction acts on a single bit **a** as part of a joint probability distribution on the values of several bits **a**, **b**, **c**, and so on. We will now explain how to describe transformations, using some of the values in a probability distribution.

We begin by again considering joint probability distributions. Suppose that we have a register **A** of  $m$  bits, which can store any of  $M = 2^m$  different values, and another register **B** of  $n$  bits, storing any of  $N = 2^n$  different values. Let  $\mathbf{p} \in \mathbb{R}^M$  be a probability vector over elements of  $\{0, 1\}^m$ , used to represent an  $M$ -element set; similarly, let  $\mathbf{q} \in \mathbb{R}^N$  be a probability vector over elements of  $\{0, 1\}^n$ , used to represent the elements of an  $N$ -element set. If the register **A** has values distributed according to  $\mathbf{p}$ , and **B** **independently** has values distributed according to  $\mathbf{q}$ , then their joint probability distribution  $\mathbf{d} \in \mathbb{R}^{M \times N}$  is described by

$$d_{x,y} = \Pr[\mathbf{A}=x \text{ and } \mathbf{B}=y] = \Pr[\mathbf{A}=x] \cdot \Pr[\mathbf{B}=y] = p_x q_y. \quad (1.46)$$

Writing the vector  $\mathbf{d} \in \mathbb{R}^{M \times N}$  as a column, with coefficients  $d_{x,y}$  listed in order of the  $m + n$  bitstring  $xy$ , we have

$$\mathbf{d} = \begin{bmatrix} p_0 q_0 \\ p_0 q_1 \\ \vdots \\ p_0 q_{N-1} \\ \hline p_1 q_0 \\ p_1 q_1 \\ \vdots \\ p_1 q_{N-1} \\ \hline \vdots \\ \hline p_{M-1} q_0 \\ p_{M-1} q_1 \\ \vdots \\ p_{M-1} q_{N-1} \end{bmatrix} = \begin{bmatrix} p_0 \mathbf{q} \\ \hline p_1 \mathbf{q} \\ \hline \vdots \\ \hline p_{M-1} \mathbf{q} \end{bmatrix} =: \mathbf{p} \otimes \mathbf{q}. \quad (1.47)$$

We use the notation  $\mathbf{p} \otimes \mathbf{q}$  to represent a probability vector which is constructed in this way. This is a block-matrix representation of  $\mathbf{d}$ , made up of  $M$  blocks which are proportionate to  $\mathbf{q}$ , scaled by the different probabilities  $p_x$ . We call this the **tensor product** of  $\mathbf{p}$  and  $\mathbf{q}$ . As each block is proportionate to  $\mathbf{q}$ , we can see that the distribution of **B** is equal to  $\mathbf{q}$  independently of what the value of **A** is.

This gives us an approach to describe transformations on the value **A** or **B**, which do not affect the value of the other register, using an operation on the joint distribution

$\mathbf{p} \otimes \mathbf{q}$  which has a similar block structure. To describe two operations  $\text{OPN}_1 \mathbf{A}$  and  $\text{OPN}_2 \mathbf{B}$  (performed in either possible order) which transform  $\mathbf{p} \mapsto S\mathbf{p}$  and  $\mathbf{q} \mapsto T\mathbf{q}$  respectively, what we would want to do if  $\mathbf{A}$  and  $\mathbf{B}$  were independent is to perform the transformation

$$\mathbf{p} \otimes \mathbf{q} \longmapsto (S\mathbf{p}) \otimes (T\mathbf{q}). \quad (1.48)$$

We can do this, if we can transform the blocks of  $\mathbf{d} = \mathbf{p} \otimes \mathbf{q}$  independently of one another. We can do this with the  $MN \times MN$  matrix

$$S \otimes T = \begin{bmatrix} S_{0,0}T & S_{0,1}T & \cdots & S_{0,M-1}T \\ S_{1,0}T & S_{1,1}T & \cdots & S_{1,M-1}T \\ \vdots & \vdots & \ddots & \vdots \\ S_{M-1,0}T & S_{M-1,1}T & \cdots & S_{M-1,M-1}T \end{bmatrix}, \quad (1.49)$$

which consists of  $N \times N$  blocks, each equal to  $T$  (scaled by a coefficient of  $S$ ). We call  $S \otimes T$  the **tensor product** of  $S$  and  $T$ : then we have  $(S \otimes T)(\mathbf{p} \otimes \mathbf{q}) = (S\mathbf{p}) \otimes (T\mathbf{q})$ .

We remark on two special cases: where either  $S$  or  $T$  is the identity operation (so that one of  $\mathbf{A}$  or  $\mathbf{B}$  is left alone). If we want to perform  $\text{OPN}_2$  on  $\mathbf{B}$  and leave  $\mathbf{A}$  alone, transforming  $\mathbf{q} \mapsto T\mathbf{q} =: \mathbf{q}'$ , the transformation which we are interested in is

$$\begin{bmatrix} p_0 \mathbf{q} \\ \text{---} \\ p_1 \mathbf{q} \\ \text{---} \\ \vdots \\ \text{---} \\ p_{M-1} \mathbf{q} \end{bmatrix} \xrightarrow{\text{OPN}_2 \mathbf{B}} \begin{bmatrix} p_0 \mathbf{q}' \\ \text{---} \\ p_1 \mathbf{q}' \\ \text{---} \\ \vdots \\ \text{---} \\ p_{M-1} \mathbf{q}' \end{bmatrix} = \begin{bmatrix} T & 0 & 0 & \cdots & 0 \\ 0 & T & 0 & \cdots & 0 \\ 0 & 0 & \ddots & & \vdots \\ \vdots & \vdots & & \ddots & 0 \\ 0 & 0 & \cdots & 0 & T \end{bmatrix} \begin{bmatrix} p_0 \mathbf{q} \\ \text{---} \\ p_1 \mathbf{q} \\ \text{---} \\ \vdots \\ \text{---} \\ p_{M-1} \mathbf{q} \end{bmatrix} \quad (1.50)$$

where on the right-hand side, we consider an  $MN \times MN$  block matrix, where we have  $M$  copies of  $T$  on the diagonal, and the  $N \times N$  zero matrix elsewhere. (As each block is proportional to the matrix  $T$ , each  $N$ -dimensional block describing the distribution of values on  $\mathbf{B}$  is transformed by  $T$ .) If we instead want to perform an  $\text{OPN}_1$  on  $\mathbf{A}$ , transforming  $\mathbf{p} \mapsto S\mathbf{p} = \mathbf{p}'$ , then the transformation which we are interested in is

$$\begin{bmatrix} p_0 \mathbf{q} \\ \text{---} \\ p_1 \mathbf{q} \\ \text{---} \\ \vdots \\ \text{---} \\ p_{M-1} \mathbf{q} \end{bmatrix} \xrightarrow{\text{OPN}_1 \mathbf{A}} \begin{bmatrix} p'_0 \mathbf{q} \\ \text{---} \\ p'_1 \mathbf{q} \\ \text{---} \\ \vdots \\ \text{---} \\ p'_{M-1} \mathbf{q} \end{bmatrix} = \begin{bmatrix} S_{0,0}I & S_{0,1}I & \cdots & S_{0,M-1}I \\ S_{1,0}I & S_{1,1}I & \cdots & S_{1,M-1}I \\ \vdots & \vdots & \ddots & \vdots \\ S_{M-1,0}I & S_{M-1,1}I & \cdots & S_{M-1,M-1}I \end{bmatrix} \begin{bmatrix} p_0 \mathbf{q} \\ \text{---} \\ p_1 \mathbf{q} \\ \text{---} \\ \vdots \\ \text{---} \\ p_{M-1} \mathbf{q} \end{bmatrix}. \quad (1.51)$$

(The fact that each block is proportional to the  $N \times N$  identity matrix  $I$  means that each block of the resulting transformation is equal to the original distribution  $\mathbf{q}$  on  $\mathbf{B}$ .)

We described  $S \otimes T$  as the way to describe operations registers, whose values have **independent** probability distributions. The good news is that the same transformations

also describe the effect of these operations on joint distributions of registers **no matter whether** those registers have independent distributions. The ‘definite’ distributions  $|(x, y)\rangle$ , which we also write as  $|x, y\rangle$  or  $|xy\rangle$ , are examples of distributions in which the values of A and B are independently distributed. As  $S \otimes T$  describes the transformation on A and B in that case, it follows that  $S \otimes T$  describes the transformation of **any** probability distribution – because the transformation matrix is determined by how it acts on the vectors  $|x, y\rangle$ .

**Example: the program of Figure 1.12.** We can consider how we can use this to describe the matrix associated with the ‘SHARED\_RANDOMNESS’ program. We can now use the operator  $C$  to describe the effect of the operation `!randomise a`, and describe the effect on the joint distribution of the values of **a** and **b**:

$$C \otimes I = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix} \otimes I = \begin{bmatrix} \frac{1}{2}I & \frac{1}{2}I \\ \frac{1}{2}I & \frac{1}{2}I \end{bmatrix} = \begin{bmatrix} 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 1/2 \\ 1/2 & 0 & 1/2 & 0 \\ 0 & 1/2 & 0 & 1/2 \end{bmatrix}. \quad (1.52)$$

The transformation performed on any joint distribution  $\mathbf{s} \in \mathbb{R}^4$  of input values, is then the result of the sequence  $\mathbf{s} \xrightarrow{C \otimes I} \mathbf{p} \xrightarrow{\text{CNOT}} \mathbf{r}$ , so that the output distribution  $\mathbf{d}''$  depends on the input distribution by  $\mathbf{r} = \text{CNOT} \cdot (C \otimes I) \cdot \mathbf{s}$ . In particular, this means that we can describe the transformation  $P$  realised by the whole procedure, in terms of the transformations described by each of the instructions, `!randomise a` and `CNOT`:

$$P = \text{CNOT} (C \otimes I), \quad (1.53)$$

where the order of the matrix multiplication is from right to left (in the order of the transformations that we perform on the vector  $\mathbf{s}$ ).

### 1.3.5 Using ‘ket’ notation to trace through RAALcr programs

Probability vectors allow us to describe what a randomised computation does, without having to have special knowledge about the problem it is trying to solve. But writing out the distribution as a column vector takes up space, and doesn’t represent the possible outcomes very directly. We can understand randomised computations more easily by consistently using ‘ket’ notation, which **explicitly** describes possible stored values of the registers. For example, for any input distribution  $\mathbf{s} \in \mathbb{R}^2$ , we have

$$C \mathbf{s} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} = \frac{1}{2} |0\rangle + \frac{1}{2} |1\rangle; \quad (1.54)$$

and for definite values of one, two or three registers, we can describe the transformation realised by any of the instructions `NOT`, `CNOT`, or `CCNOT` by

$$\text{NOT} |x\rangle = |\neg x\rangle, \quad \text{CNOT} |x, y\rangle = |x, y \oplus x\rangle, \quad \text{CCNOT} |x, y, r\rangle = |x, y, r \oplus (x \cdot y)\rangle. \quad (1.55)$$

Because the effect of any randomised procedure is given by a matrix multiplication, we may distribute the matrix multiplication over any weighted sum of terms produced by randomising bit-values.

**Example: the program of Figure 1.12.** Let's again describe the effect of the 'SHARED\_RANDOMNESS' program, considering the case where **a** and **b** both initially have the value 0. We may represent the sequence of transformations performed by the instructions as follows:

$$\begin{aligned} |0, 0\rangle &\xrightarrow{C \otimes I} \left(\frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle\right) \otimes |0\rangle \\ &= \frac{1}{2}|0, 0\rangle + \frac{1}{2}|1, 0\rangle \xrightarrow{\text{CNOT}} \frac{1}{2}|0, 0\rangle + \frac{1}{2}|1, 1\rangle \end{aligned} \quad (1.56)$$

which clearly represents that the first operation randomises the first bit register **a** (leaving **b** with the value  $|0\rangle$ ), and that the second operation maps the pair of registers to a random but equal value (either both 0 or both 1).

### 1.3.6 Conditional probabilities, projectors, and probability of failure

For two variables / registers at a given point in time, we can consider the probabilities of the values of one register, depending on the value of another: we can define the probability that a register **A** takes some particular value  $x \in \{0, 1\}^m$ , **conditioned** on another register **B** taking some particular value  $y \in \{0, 1\}^n$ , by

$$\Pr[A=x \mid B=y] = \frac{\Pr[A=x \text{ and } B=y]}{\Pr[B=y]} \quad (1.57)$$

provided that  $\Pr[B=y] > 0$ . The reason for dividing by  $\Pr[B=y]$  is that we effectively consider only the part of the probability distribution in which  $B=y$ , discarding those outcomes in which this does not occur. (If the total probability of these outcomes is less than 1, we divide by  $\Pr[B=y]$  to obtain a distribution of possibilities which sum to 1.)

To describe how to compute this in practical cases, we wish to describe how to represent a projector onto a particular vector  $|y\rangle$ , representing a fixed value of the register **B** in a probability distribution.

To describe these projectors, we define a counterpart to the 'ket' vectors  $|x\rangle$ , which describe row vectors instead of column vectors. We let  $\langle x|$  – pronounced 'bra- $x$ ' – describe the row vector  $|x\rangle^\top$ . (When we later consider vectors with complex coefficients, the row-vector  $\langle v|$  will not necessarily be just the transpose of  $|v\rangle$ ; but for a real-valued vector  $|x\rangle$  as we have here, we do have  $\langle x| = |x\rangle^\top$ .) For instance, for  $x \in \{0, 1\}$ , we have

$$\langle 0| = [1 \ 0], \quad \langle 1| = [0 \ 1]; \quad (1.58)$$

for  $x \in \{00, 01, 10, 11\}$ , we have

$$\langle 00| = [1 \ 0 \ 0 \ 0], \quad \langle 01| = [0 \ 1 \ 0 \ 0], \quad \langle 10| = [0 \ 0 \ 1 \ 0], \quad \langle 11| = [0 \ 0 \ 0 \ 1]; \quad (1.59)$$

and so forth. We define these, so that we can multiply a 'bra' vector  $\langle x|$  with a 'ket' vector  $|y\rangle$ , to get a **bracket**,  $\langle x|y\rangle$  — which is the inner product of the two 'ket' vectors  $|x\rangle$  and  $|y\rangle$ , and is meant to resemble the common notation  $\langle \mathbf{u}, \mathbf{v} \rangle$  for inner products of

vectors  $\mathbf{u}$  and  $\mathbf{v}$ . So, we have  $\langle 0|0\rangle = \langle 1|1\rangle = 1$  and  $\langle 0|1\rangle = \langle 1|0\rangle = 0$ . More generally, for  $x, y \in \{0, 1\}^n$ , we have

$$\langle x|y\rangle = \begin{cases} 1, & \text{if } x = y; \\ 0, & \text{otherwise.} \end{cases} \quad (1.60)$$

Having defined the ‘bra-vectors’  $\langle x|$ , we may define orthogonal projection operations  $|x\rangle\langle x|$ , consisting of a column vector multiplied by a row vector, which projects onto the  $|x\rangle$  component of a vector. For instance, for  $x \in \{0, 1\}$ , we have

$$|0\rangle\langle 0| = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad |1\rangle\langle 1| = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}; \quad (1.61)$$

for  $x \in \{00, 01, 10, 11\}$ , we have

$$|00\rangle\langle 00| = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad |01\rangle\langle 01| = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad |10\rangle\langle 10| = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \text{etc.}; \quad (1.62)$$

and so forth. We can describe these operators as ‘selecting’ components of probability vectors which are consistent with the value described in the projector, and ‘discarding’ or ‘ignoring’ the rest.

**Example: a quick-and-dirty dice roll simulator.** In RAALcr, with just the one instruction `!randomise` to generate a random bit, we can still simulate other probability distributions – though with some chance of failure. For instance, to simulate rolling a six-sided die with outcomes  $\{1, 2, 3, 4, 5, 6\}$ , we might flip three bits to obtain a random bitstring  $x_2x_1x_0 \in \{0, 1\}^3$  representing an integer  $0 \leq x \leq 7$ , and accept the result as representing a die-roll only if the outcome is not 000 or 111 (representing the integers 0 and 7). We demonstrate such a procedure in Figure 1.14. The probability of failure of this procedure is  $2/8 = 1/4$  (the chance of obtaining either 0 or 7 when producing an integer  $0 \leq x \leq 7$  at random). This procedure uses the subroutines `P_AND3` to evaluate whether the first three bits are all 1, and if so, to ‘toggle’ the bit indicating failure.

Consider the effect of `TRY_DIE_ROLL` on an input of  $(\mathbf{r}, \mathbf{f})$  with initial value  $(000, 0)$ , represented by the distribution  $|000, 0\rangle$ . (This is a 16-dimensional vector: by using ket notation, we both avoid having to write 16-dimensional vectors and we will also see more explicitly what the possible outcomes are at each stage of the computation.) We may describe the effect of `TRY_DIE_ROLL`  $\mathbf{r}, \mathbf{f}$  one instruction at a time, as follows. The first step randomises the first bit:

$$|000, 0\rangle \xrightarrow{C \otimes I \otimes I \otimes I} \frac{1}{2}(|000, 0\rangle + |100, 0\rangle).$$



Figure 1.14: The RAALcr procedure ‘TRY\_DIE\_ROLL’. This procedure produces a three bit register  $r$  indicating an integer in  $\{0, 1, \dots, 7\}$ , and a one-bit register indicating whether the procedure ‘failed’ (in other words, whether the integer generated is 0 or 7).

```

define TRY_DIE_ROLL r, f
  # attempt to generate an integer 1-6 as a three-bitstring in ‘r’,
  # with a flag ‘f’ which we ‘toggle’ if the procedure fails
  operand register r: 3 bits
  operand bit f

  # generate random bits (which we will logically negate below)
  !randomise r[0]           # randomise value of unit bit
  !randomise r[1]           # randomise value of twos bit
  !randomise r[2]           # randomise value of fours bit

  P_AND3 r[0], r[1], r[2], f # test if the outcome is ‘000’
  NOT r[0]                   # negate units place
  NOT r[1]                   # negate twos place
  NOT r[2]                   # negate fours place
  P_AND3 r[0], r[1], r[2], f # test if the outcome is ‘111’
  stop

```

After this step, the first bit could be either 0 or 1, and the other bits remain 0. The possible values of the registers are then  $(000, 0)$  and  $(100, 0)$ , each with probability  $\frac{1}{2}$ . The two possible values are shown explicitly in the ket vectors; to simplify things, we factorise it as a common factor. The possible values of the registers change in a similar way when we randomise the second and third bits as well:

$$\begin{aligned}
& \xrightarrow{I \otimes C \otimes I \otimes I} \frac{1}{4} \left( |000, 0\rangle + |010, 0\rangle + |100, 0\rangle + |110, 0\rangle \right) \\
& \xrightarrow{I \otimes I \otimes C \otimes I} \frac{1}{8} \left( |000, 0\rangle + |001, 0\rangle + |010, 0\rangle + |011, 0\rangle + |100, 0\rangle + |101, 0\rangle + |110, 0\rangle + |111, 0\rangle \right).
\end{aligned}$$

The next step determines whether all of the first three bits are 1, and toggles the final bit if so. (The three NOT instructions that follow, indicate whether the first three bits are 0 instead.) We can describe these operations by their effects on each term independently.

$$\begin{aligned}
& \xrightarrow{P\_AND3} \frac{1}{8} \left( |000, 0\rangle + |001, 0\rangle + |010, 0\rangle + |011, 0\rangle + |100, 0\rangle + |101, 0\rangle + |110, 0\rangle + |111, 1\rangle \right) \\
& \xrightarrow{NOT \otimes I \otimes I \otimes I} \frac{1}{8} \left( |100, 0\rangle + |101, 0\rangle + |110, 0\rangle + |111, 0\rangle + |000, 0\rangle + |001, 0\rangle + |010, 0\rangle + |011, 1\rangle \right) \\
& \xrightarrow{I \otimes NOT \otimes I \otimes I} \frac{1}{8} \left( |110, 0\rangle + |111, 0\rangle + |100, 0\rangle + |101, 0\rangle + |010, 0\rangle + |011, 0\rangle + |000, 0\rangle + |001, 1\rangle \right) \\
& \xrightarrow{I \otimes I \otimes NOT \otimes I} \frac{1}{8} \left( |111, 0\rangle + |110, 0\rangle + |101, 0\rangle + |100, 0\rangle + |011, 0\rangle + |010, 0\rangle + |001, 0\rangle + |000, 1\rangle \right).
\end{aligned}$$

The next step again determines whether all of the first three bits are 1 (after the NOT operations), and if so, toggles the final bit. As this is the final instruction, the final bit is 1 if, and only if, the first three bits are 000 or 111.

$$\xrightarrow{\text{P\_AND3}} \frac{1}{8} \left( |111,1\rangle + |110,0\rangle + |101,0\rangle + |100,0\rangle + |011,0\rangle + |010,0\rangle + |001,0\rangle + |000,1\rangle \right).$$

Let  $\mathbf{d}$  be this final probability distribution. We wish to consider the ‘sub-distribution’ in which  $\mathbf{f} = 0$ , representing the situation where we succeed in simulating a 6-sided die roll. To consider only the terms in which  $\mathbf{f} = 0$ , we may consider the projector  $P = |0\rangle\langle 0|$ , and apply this to the values of the register  $\mathbf{f}$ : this yields a (subnormalised) distribution

$$\begin{aligned} \tilde{\mathbf{q}} &= (I \otimes I \otimes I \otimes P) \mathbf{d} \\ &= \frac{1}{8} |001,0\rangle + \frac{1}{8} |010,0\rangle + \frac{1}{8} |011,0\rangle + \frac{1}{8} |100,0\rangle + \frac{1}{8} |101,0\rangle + \frac{1}{8} |110,0\rangle, \end{aligned} \quad (1.64)$$

consisting of just those terms in  $\mathbf{d}$  for which  $\mathbf{f}$  is zero. The total of the coefficients of  $\tilde{\mathbf{q}}$ , is  $p = 3/4$ : this implies that  $p = \Pr[\mathbf{f} = 0] = \frac{3}{4}$ . The conditional distribution  $\mathbf{q}$  of the two registers  $\mathbf{r}$  and  $\mathbf{f}$ , given that  $\mathbf{f}$  is zero, is then

$$\mathbf{q} = \frac{1}{p} \tilde{\mathbf{q}} = \frac{1}{6} |001,0\rangle + \frac{1}{6} |010,0\rangle + \frac{1}{6} |011,0\rangle + \frac{1}{6} |100,0\rangle + \frac{1}{6} |101,0\rangle + \frac{1}{6} |110,0\rangle, \quad (1.65)$$

where every term satisfies  $\mathbf{f} = 0$ . Thus, if TRY\_DIE\_ROLL succeeds, the result is a uniformly random integer from 1–6 represented in binary (each occurring with a probability of 1/6).

### 1.3.7 The power of repetition, and control statements in RAALcr

If a randomised algorithm fails, and we **know** that it failed, we can apply an old piece of advice: **if at first you don’t succeed, try, try again**. In the setting of the RAALcr language, we have a control system which can determine what operations do next. If the control system can determine that a subroutine has ‘failed’, it can attempt to perform the procedure again. If the probability of failure is not too large, we will eventually succeed after only a reasonable number of repetitions.

We define declarations and instructions for the control system in RAALcr, to facilitate such repetition:

```
@label:                # a label to jump to, in the instructions
!integer n := value     # declaration of variable in control system
!set n := expr          # re-assignment of value for control variable
!jump @label            # jump to the instruction at @label
!jump_if_zero reg, @label # jump to @label, if variable/register is 0
!jump_if_gtr reg, val, @label # jump to @label, if var/register > val
```

**Example: improved dice roll simulators.** For TRY\_DIE\_ROLL as defined in Figure 1.14, the probability of failure is  $\frac{1}{4}$ . Then the probability of it failing  $k$  times in succession, is  $(\frac{1}{4})^k = 2^{-2k}$ , which shrinks very quickly: if it succeeds on any of those times, the result is a successful simulation of a die roll. We only have to repeat TRY\_DIE\_ROLL

Figure 1.15: The RAALcr procedures ‘DIE\_ROLL’ and ‘REALLY\_TRY\_DIE\_ROLL’. The first of these procedures produces a three bit register *r* storing an integer in  $\{1, 2, \dots, 6\}$ , by repeating TRY\_DIE\_ROLL until it succeeds. The second procedure attempts (at most) a fixed number of times, accepting some small probability of failure.

```
define DIE_ROLL r
  # generate an integer 1--6 as a three-bit string stored in r
  operand register r: 3 bits
  bit f # a register to use as an indicator for failure

  @repeat:                                # a label for the start of a loop
    TRY_DIE_ROLL r, f                    # attempt to generate a die roll
    !jump_if_zero f, @done               # escape the loop if f is zero
    NOT f                                # reset the value of f to zero
    !jump @repeat                        # repeat the loop

  @done:                                  # a label to jump to in case of success
  stop

define REALLY_TRY_DIE_ROLL r, f
  # attempt to generate an integer 1--6 as a three-bit string
  # stored in r, with a flag f which we ‘toggle’ if we fail
  operand register r: 3 bits
  operand bit f
  !integer n := 20                        # limit on the number of iterations

  @repeat:                                # a label to jump to in case of failure
    !set n := n-1                        # decrease the value of n
    TRY_DIE_ROLL r, f                    # attempt to generate a die roll
    !jump_if_zero f, @done               # escape the loop if f is zero
    !jump_if_zero n, @done               # escape the loop if n is zero
    NOT f                                # reset the value of f to zero
    !jump @repeat                        # repeat the loop

  @done:                                  # a label to jump to to escape the loop
  stop
```

at most ten times, to have a probability of success better than 99.9999%; after twenty times, the probability of failure is less than one in a trillion. We may then define a procedure which keeps trying to simulate rolling a 6-sided die until it succeeds, as presented in the procedure DIE\_ROLL in Figure 1.15. This procedure keeps on jumping back to the label @repeat until the work-bit *f* contains the value 0 after calling TRY\_DIE\_ROLL. Once it succeeds, we know from the analysis above that the result in *r* will be a bitstring representing a uniformly random integer 1–6.

In principle, there is no upper bound on the number of times that the procedure DIE\_ROLL may need to repeat the loop – we cannot rule out the possibility of freakishly bad luck.

If we wanted to absolutely guarantee some upper limit to the number of iterations that we take, we must settle for some probability of failure, however small. We can do this by having the control system maintain a counter, and stopping if the counter drops to zero. Such a procedure is presented as `REALLY_TRY_DIE_ROLL` in Figure 1.15.

## Summary

Well done! You have now reached the end of this week's Study section.

This week, we introduced concepts which will be essential for the rest of the module, but which are not specifically about quantum computation.

- Boolean logic underlies all of low-level digital computation: understanding this is essential to understanding the other two topics – reversible computation and randomised computation.
- Reversible computation captures a way of computing functions, which reflects some of the constraints of quantum computation (whose operations are for the most part **also** reversible).
- Randomised computation provides us with a way of understanding how to compute with an element of chance, and, in particular, ways that we can describe them mathematically on an instruction-by-instruction basis.

We have also taken the opportunity to introduce `RAAL` and `RAALcr`, which are assembly-like languages to describe reversible and randomised computation. We will build on this method of describing procedures as simply as possible in order to describe quantum computational procedures.

## References

- [1] Nielsen, M. A. and Chuang, I. L. Quantum Computation and Quantum Information. 10th anniversary edn., 2010, p. 159
- [2] Toffoli, T. Reversible computing. Automata, Languages and Programming, Seventh Colloquium, 1980, pp. 632–644. doi:10.1007/3-540-10003-2\_104